PassProbe: Quantifying Compiler Pass Contributions in Quantum Transpilation

Prateek P. Kulkarni
PES University, India
pkulkarni2425@gmail.com

***Abstract*—Modern quantum transpilers use pipelines of multiple named passes (layout, routing, cancellation, scheduling), yet it is unclear which passes are truly critical and whether this depends on workload. We present PASSPROBE, which ablates all 2^k pass subsets in a k-pass pipeline and applies Shapley-value attribution to measure each pass's marginal contribution to circuit quality. Across 20 circuits, six workload classes, and four state-of-the-art compilers, we find that just 2–3 passes drive over 80% of quality gains, with the dominant pass shifting predictably by workload; pass ordering effects can exceed selection effects for deep circuits; and 3–5 passes per compiler can be safely pruned, cutting compilation time by up to 22% with <0.5% loss in fidelity.**

I. Introduction

Logical quantum circuits cannot run directly on current hardware. Limited connectivity, restricted native gates, and uneven noise require compilers to transform circuits through sequential passes such as routing, gate decomposition, and scheduling. Modern transpilers chain 10–20+ passes: Qiskit uses 12 for layout and routing [1], tket 10 for decomposition and peephole optimization [2], Cirq 8 for gate merging [3], and BQSKit 9 for numerical synthesis [4]. Circuit quality therefore emerges from the entire pipeline, not any single pass. This raises two questions: *which passes matter most*, and *does their importance depend on circuit structure?* Despite extensive work on individual passes and end-to-end benchmarking [5], no prior study quantifies each pass's marginal contribution *within* the full pipeline.

To address this, we propose PASSPROBE, which (drawing inspiration from cooperative game theory) models compiler passes as players and applies the Shapley value [6] to attribute pipeline outcomes. For a k-pass pipeline (Figure 1), PASSPROBE evaluates all 2^k pass subsets and computes each pass's average marginal contribution to expected fidelity[1]. **First**, we cast pass attribution as an exact combinatorial accounting problem rather than an ablation heuristic, evaluating the full subset lattice instead of relying on greedy or one-pass-at-a-time removal. **Second**, we evaluate this lattice to rigorously quantify each pass's marginal contribution to expected circuit fidelity, a strong proxy for NISQ performance. **Third**, we repeat this procedure across four compilers and a representative hardware topology, enabling direct comparison of pass attribution at the level of functional categories rather than compiler-specific pass names.

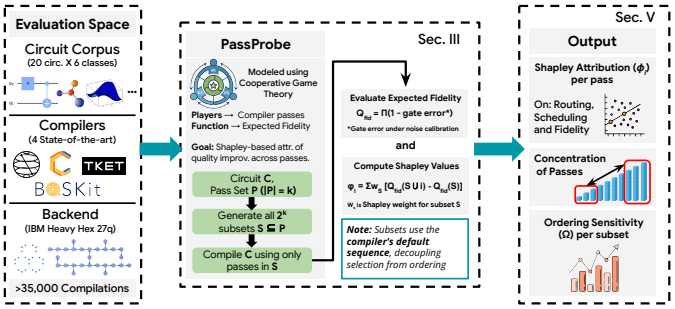


Fig. 1: PASSPROBE framework overview. For each (circuit, compiler, topology) triple, it exhaustively evaluates 2^k pass subsets, measuing expected fidelity and applies Shapley attribution to quantify pass contributions, concentration, and ordering sensitivity.

Our results show substantial structure in how pipelines accumulate quality improvement: just 2–3 passes account for over 80% of total quality improvement across all workload classes and compilers, and which pass dominates shifts predictably with workload structure. These results hold across all four compilers and translate directly into pruning guidance without sacrificing circuit quality. Our key contributions are:

1) **Systematic pass attribution.** We propose PASSPROBE, a framework to quantify the marginal contribution of individual passes within the full transpilation pipeline across heterogeneous compiler architectures.
2) **Shapley formulation.** We model pass attribution as a cooperative game over the 2^k subset lattice, using Shapley values to provide a unique, mathematically rigorous allocation of quality improvement.
3) **Empirical evaluation at scale.** Across 20 circuits, six workload classes, four compilers, and a representative topology (over 35,000 compilations), we reveal workload-dependent pass dominance, cross-compiler consistency, and ordering effects that exceed pass selection.

II. Background and Related Work

Quantum Transpilation Pipelines. A quantum transpiler converts a logical circuit into one that real hardware can run, using a sequence of *passes* [1]. These fall into four categories:

- **Layout and routing:** To place logical qubits on physical qubits and insert SWAPs where needed.
- **Gate cancellation and optimization:** To remove redundant gates and merge or rewrite single-qubit gates.

[1]Statevector simulation is intractable at scale (tracking $2^{\#\text{qubits}}$ complex amplitudes), hence we use the expected fidelity instead, which is standard in literature. See for example [5], [7], [8].

- **Basis translation and synthesis:** To decompose gates into the hardware's native gate set.
- **Scheduling:** To order operations to reduce idle time and decoherence.

Each compiler implements these with its own passes: routing via SabreSwap [9], RoutingPass [2], or QSearchRouting [4]; optimization via CXCancellation, CliffordSimp [10], or MergeInteractions; basis translation via BasisTranslator, UnitarySynthesis, or QFactorInstantiation [4]; and scheduling via passes like ALAPSchedule. Passes interact: routing adds SWAPs that optimization may later remove, and synthesis can bypass routing altogether for small blocks. A pass's value therefore depends on which other passes are present, not on the pass alone – which is why we evaluate passes jointly, as a full 2^k coalition, rather than one at a time.

Related Work. Most prior work improves individual passes in isolation: routing via heuristics like SABRE [9], hardness results and tket's routing scheme [11], and optimal mapping [12]; gate optimization via template matching [10], ZX-calculus [13], and synthesis-driven compilation [4]. These works make specific passes better, but none measures how much a pass actually contributes once it's part of a full pipeline.

Other work benchmarks compilers end-to-end as black boxes, e.g., MQT Bench [5] and BenchPress [14]. This tells us which compiler produces a better final circuit, but not which pass inside it was responsible. Prior work has also extensively characterized isolated routing and SWAP insertion [9], [11], [12]. Since these standalone metrics are well understood, we abstract them away to focus exclusively on holistic pipeline interactions and fidelity.

Shapley values [6] are a standard tool for attribution in game theory and ML interpretability [15], but have not been used to analyze quantum compilers. PASSPROBE fills this gap: instead of improving one pass or benchmarking a whole pipeline, it measures the marginal contribution of every pass within the pipeline, using a principled allocation that works across compilers with very different pass architectures.

III. THE PASSPROBE FRAMEWORK

We first illustrate the pass attribution problem with an example, and then formalize it as a coalition-game attribution problem over the 2^k pass-subset lattice.

Motivating Example. Consider a 14-qubit QAOA circuit compiled on Heavy-hex via Qiskit's level-3 pipeline, reaching an expected fidelity of 0.384. Removing SabreSwap breaks hardware compliance (fidelity ≈ 0), while removing CXCancellation or ALAPSchedule drops fidelity by 2% and 4%. This highlights two attribution challenges: passes interact nonlinearly (e.g., routing increases depth and noise, which cancellation later mitigates), and a pass's value depends heavily on the presence of others. PASSPROBE captures these interdependencies by averaging each pass's marginal contribution to expected fidelity across all pipeline configurations.

Quality Metric. Let $\mathcal{P} = \{p_1, p_2, \ldots, p_k\}$ denote the k ablatable passes of a compiler pipeline, and let $\mathcal{C}$ denote an input circuit. For any subset $S \subseteq \mathcal{P}$, we define a quality function $Q(\mathcal{C}, S) : 2^{\mathcal{P}} \to \mathbb{R}$, which scores the compiled circuit obtained by applying only the passes in S in a fixed canonical order. We instantiate Q using expected fidelity as the proxy for NISQ performance: $Q_{\text{fid}}(\mathcal{C}, S) = \prod_{g \in G}(1 - \epsilon_g)$, where G is the set of gates in the compiled circuit and ϵ_g is the error rate of gate g under a calibrated noise model. Focusing strictly on fidelity allows us to directly measure the passes that most critically preserve quantum information against decoherence and gate errors.

Shapley Value Attribution. We now formally define pass attribution as a cooperative game in which each pass is a player and Q is the characteristic function. The Shapley value of pass p_i quantifies its average marginal contribution across all possible subsets of the remaining passes:

$$\phi_i(\mathcal{C}) = \sum_{S \subseteq \mathcal{P} \setminus \{p_i\}} \frac{|S|!\,(k-|S|-1)!}{k!} \left[Q(\mathcal{C}, S \cup \{p_i\}) - Q(\mathcal{C}, S)\right]$$

Intuitively, ϕ_i averages the marginal quality contribution of pass p_i across all possible coalitions of other passes, weighted by the probability of each coalition forming in a random ordering. Three axioms [6] make this ideal for our setting: (1) *Efficiency* guarantees $\sum_{i=1}^{k} \phi_i = Q(\mathcal{C}, \mathcal{P}) - Q(\mathcal{C}, \emptyset)$, exactly partitioning the total quality gain with no residual; (2) *Symmetry* ensures passes with identical marginal effects receive equal scores, enabling cross-compiler comparison; and (3) the *null-player* axiom assigns zero to passes that contribute nothing, grounding our pruning criterion.

With $k \leq 12$ passes per compiler, exhaustive enumeration of all 2^k subsets is highly tractable (at most 4,096 evaluations per circuit–compiler–topology triple, taking ∼0.1–2 seconds each in most cases). This allows exact Shapley computation across our full study without relying on Monte Carlo approximations.

Derived Metrics. Three derived quantities translate the raw Shapley values into the comparisons reported in Section V:

- ***Normalized attribution.*** To compare attribution across circuits of differing absolute quality improvement, we normalize each pass's Shapley value by the total absolute attribution in that circuit: $\hat{\phi}_i(\mathcal{C}) = |\phi_i(\mathcal{C})| / \sum_{j=1}^{k} |\phi_j(\mathcal{C})|$.
- ***Concentration ratio.*** To quantify how much of the total attribution is captured by a small number of passes, we define $\text{CR}_m(\mathcal{C}) = \sum_{j=1}^{m} \hat{\phi}_{(j)}(\mathcal{C})$, where $\hat{\phi}_{(j)}$ denotes the j-th largest normalized attribution. We call a pass *dominant* if $\hat{\phi}_i > 0.25$ (it alone accounts for over a quarter of total quality improvement) and *negligible* if $\hat{\phi}_i < 0.02$ (below the noise floor of our fidelity measurements).
- ***Ordering sensitivity***. Shapley attribution decomposes contribution by pass *selection*, but does not by itself capture sensitivity to pass *ordering*. To disentangle the two, we define, for a fixed subset S,

$$\Omega(S, \mathcal{C}) = \frac{\max_\pi Q(\mathcal{C}, \pi(S)) - \min_\pi Q(\mathcal{C}, \pi(S))}{Q(\mathcal{C}, \mathcal{P})}$$

where π ranges over permutations of S; we sample $\min(100, |S|!)$ random permutations when $|S| > 5$ to keep this computation tractable. We formulate the pass attribution problem as follows: "*Given a compiler pipeline with pass set $\mathcal{P}$, an input circuit $\mathcal{C}$, and a quality function Q, compute*

a fair allocation $\{\phi_i(\mathcal{C})\}_{i=1}^{k}$ *of the total quality improvement* $Q(\mathcal{C},\mathcal{P}) - Q(\mathcal{C},\emptyset)$ *to individual passes, together with the ordering sensitivity* $\Omega(\mathcal{P},\mathcal{C})$*, such that the allocation satisfies efficiency, symmetry, and the null-player axiom.*" This frames attribution as a combinatorial problem over the subset lattice, rather than single-pass ablation.

IV. Experimental Setup

Circuit corpus. We evaluate a diverse set of 20 circuits across 6 workload classes to capture how circuit structure influences compilation (Table I).

Compilers and passes. We test four open-source compilers at their highest optimization levels: Qiskit v2.3 [1] (10 of 12 ablatable passes, e.g., SabreSwap, BasisTranslator), tket v2.18 [2] (8 of 10, e.g., RoutingPass, CliffordSimp), Cirq v1.6 [3] (6 of 8, e.g., MergeInteractions, RouteCQC), and BQSKit v1.2 [4] (7 of 9, e.g., QSearchRouting, QFactorInstantiation). BQSKit's synthesis-driven architecture contrasts with the rule-based pipelines of the others.

Topology and noise. We target the 27-qubit Heavy-hex topology of the **ibm_cairo** backend [1] (avg. degree 2.37), representing the stringent sparse-connectivity constraints of modern superconducting hardware. Noise models use gate error rates obtained from corresponding hardware calibrations.

Evaluation protocol. For each circuit-compiler-topology combination, we exhaustively evaluate all $2^{k'}$ subsets of the k' ablatable passes, recording Q_{fid}. This protocol requires 35,840 compilations in total, executed on a standard workstation (Intel Core i7-13700K, 16 cores) in approximately 6 hours, with a median time of 1–2s per compilation and long-tail synthesis tasks taking up to a few minutes.

TABLE I: 20 benchmarks across six workload classes from [5]. CX count and depth refer to the uncompiled logical circuit.

Class	Circuit	Qubits	CX	Depth	Class	Circuit	Qubits	CX	Depth
VQE	VQE-H_2 (UCCSD)	4	52	38	BV/QFFT	BV-10	11	10	2
	VQE-LiH	10	284	112		BV-20	21	20	2
	VQE-H_2O	12	396	148		QFFT-8	8	84	56
	VQE-hardware-eff.	16	120	32	Chem.	H-chain-4	8	196	88
QAOA	QAOA-MaxCut-8	8	28	8		H-chain-6	12	448	132
	QAOA-MaxCut-14	14	62	12		BeH_2	14	562	168
	QAOA-TSP-6	12	132	26		N_2 (STO-3G)	16	724	204
QPE	QPE-4	5	46	34	Random	Rand-Struct-8	8	64	24
	QPE-8	9	184	72		Rand-Struct-14	14	168	42
	QPE-12	13	412	108		Rand-Struct-20	20	310	58

Chemistry benchmarks are generated using Qiskit Nature [1], [16].

V. Evaluation

We organize our evaluation around 4 questions, each targeting a claim about how compiler passes contribute to circuit quality. (1) **Are Pass Contributions Concentrated?** To determine if pass importance is concentrated, Figure 3 reports the concentration ratio CR_m for m top passes (aggregated across all four compilers). Across all workload classes, a small subset of passes drives nearly all quality improvements, rendering the rest negligible (averaging 3.9 of 10 ablatable passes/compiler). Even BQSKit's synthesis-driven architecture follows this trend; its lowest concentration ($\mathrm{CR}_3 = 0.78$ for chemistry workloads) occurs only because UnitarySynth distributes effects across multiple blocks rather than acting as a single dominant step.

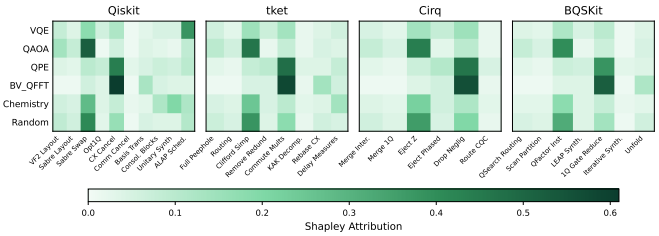


Fig. 2: Shapley attribution by pass category (heavy-hex, Q_{fid}).

Finding 1: Pass contributions are concentrated

Across all workloads and compilers, the top 3 passes capture at least 78% of total quality improvement ($\mathrm{CR}_3 \geq 0.78$), rendering most pipeline passes near-redundant.

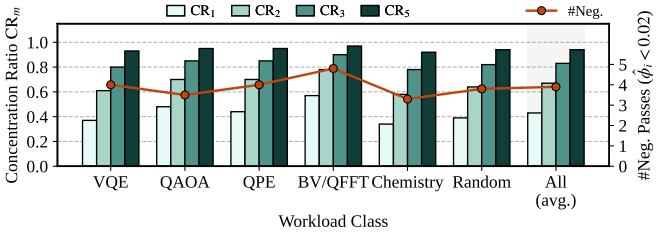


Fig. 3: Shapley concentration on heavy-hex (Q_{fid}, averaged). Bars: CR_m; Line: negligible passes ($\hat{\phi}_i < 0.02$). Top 3 passes capture $> 80\%$ of attribution.

(2) **Which Passes Dominate, and Does It Depend on the Workload?** Next, we identify which passes dominate and whether this depends on the workload. Figure 2 groups passes by functional category to enable direct cross-compiler comparison of their normalized Shapley attributions. A clear block-diagonal pattern emerges. Routing dominates QAOA due to dense qubit interactions requiring many SWAPs. Gate cancellation dominates QPE because regular controlled-rotation ladders offer extensive cancellation opportunities. Scheduling dominates VQE, as parameter sweeps create long idle windows that amplify decoherence. BQSKit is the sole exception: for chemistry workloads, numerical resynthesis (QFactorInstantiation) overtakes routing.

Finding 2: Dominance is workload-dependent

The dominant pass category is determined by the workload, not the compiler: routing for QAOA, cancellation for QPE, and scheduling for VQE. BQSKit's synthesis-based architecture is the sole exception for chemistry circuits.

Structural Drivers of Pass Dominance. To predict this dominance from lightweight features, we first tested simple metrics like CX count and depth, finding insignificant correlations (Pearson $r < 0.3$, $p > 0.2$). However, characterizing workloads by algorithmic structure yields strong patterns. For E, the set of two-qubit gate interactions (edges of the logical interaction graph), V, the set of logical qubits (vertices of the logical interaction graph), D, total depth of the uncompiled logical circuit and $g(v)$, the number of gate-occupied

cycles for qubit v, we define a ***logical connectivity score*** $\mathcal{C} = |E|/\binom{|V|}{2}$ (the edge density of the logical interaction graph), quantifying the topological stress placed on the routing pass, and an ***idle-time potential score*** $\mathcal{I} = 1 - \frac{\sum_{v \in V} g(v)}{|V|D}$ (the fraction of idle qubit-cycles), capturing the circuit's vulnerability to decoherence before scheduling. As shown in Figure 4, Routing attribution strongly correlates with logical connectivity ($r = 0.92$), explaining why highly entangled (but low-CX) QAOA dominates routing. Similarly, Scheduling ($r = 0.94$) correlates with idle-time potential driven by the parameter sweeps, confirming that lightweight structural features can accurately predict pass contributions, enabling workload-aware pipelines without exhaustive profiling.

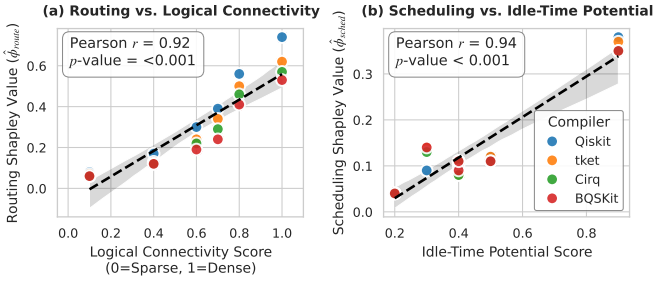


Fig. 4: (a) Routing attribution strongly correlates with logical connectivity capturing connectivity stress rather than raw gate count. (b) Scheduling attribution strongly correlates with idle-time potential driven by parameter-sweeps, across all benchmarks.

③ **Does Pass Ordering Matter More Or Pass Selection?** To determine whether selecting or ordering passes yields greater benefits, we quantify ordering sensitivity (Ω) across pass subsets and workload classes (Figure 5(a)). For quantum chemistry circuits, $\Omega_{\text{all}} > 1$, meaning the quality variance caused by pass ordering exceeds the average quality gain of the passes themselves. This stems from strong pass interactions in deep, irregular circuits, where routing dictates available cancellations and subsequent synthesis opportunities. Conversely, shallow, regular circuits (e.g., BV/QFFT) exhibit weak interactions and minimal ordering sensitivity ($\Omega < 0.25$).

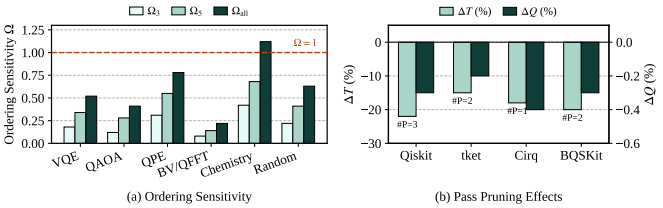
Finding 3: Ordering dominates selection for deep circuits

For quantum chemistry, ordering variance exceeds selection gains ($\Omega_{\text{all}} = 1.12$) due to strong pass interactions in deep, irregular circuits. Shallow, regular circuits show minimal sensitivity ($\Omega < 0.25$).

④ **Can We Safely Prune Negligible Passes?** Building on Finding 1, we identify passes with negligible impact ($\hat{\phi}_i < 0.02$ in $> 90\%$ of circuit-topology pairs) and measure the effect of removing them. The pruned passes include Qiskit's TrivialLayout, CommutativeCancellation, and PadDynamicalDecoupling; tket's DelayMeasures and SimplifyInitial; Cirq's EjectPhasedPaulis; and BQSKit's UnfoldPass and SingleQubitGateReduction. As shown in Figure 5(b), removing these 1–3 passes per compiler reduces compilation time by 15–22%, while fidelity drops by less than 0.5% (within noise limits). $\Delta T\%$ and $\Delta Q\%$ denote percentage difference in compilation time and fidelity (Q_{fid}) respectively, after pruning.

Finding 4: Negligible passes can be safely pruned

Removing 1–3 negligible passes per compiler cuts compilation time by 15–22% with less than 0.5% fidelity loss, directly accelerating iterative workflows like variational sweeps.



(a) Ordering Sensitivity (b) Pass Pruning Effects

Fig. 5: Ordering sensitivity averaged across all compilers on heavy hex and pass pruning effects, for Q_{fid}.

VI. Conclusion

Our results show that lightweight, workload-aware quantum compiler design is highly practical: by mapping compilation effort across different compilers, PassProbe reveals that a small subset of passes drives most quality improvements, allowing developers to skip unnecessary steps for major speedups without sacrificing performance. PassProbe thus provides a cross-compiler guide for accelerating software compilation and informing hardware-software co-design. Although our evaluation targets existing compilers and benchmark workloads, the methodology itself is compiler-agnostic and can be applied as transpilation frameworks continue to evolve.

References

[1] A. Javadi-Abhari *et al.*, "Quantum computing with Qiskit," in *arXiv preprint arXiv:2405.08810*, 2024.

[2] S. Sivarajah *et al.*, "tket: A retargetable compiler for NISQ devices," in *Quantum Science and Technology*, vol. 6, no. 1. IOP Publishing, 2020.

[3] Cirq Developers, "Cirq: A python framework for creating, editing, and invoking noisy intermediate scale quantum circuits," 2023. [Online]. Available: https://quantumai.google/cirq

[4] E. Younis *et al.*, "QFAST: Conflating circuits using language transformations for near-optimal synthesis & compilation," in *Proceedings of the 2021 IEEE QCE*.

[5] N. Quetschlich *et al.*, "MQT Bench: Benchmarking software and design automation tools for quantum computing," in *Quantum*, vol. 7, 2023.

[6] L. S. Shapley, "A value for n-person games," in *Contributions to the Theory of Games*, vol. 2. Princeton University Press, 1953.

[7] S. Nishio *et al.*, "Extracting success from ibm's 20-qubit machines using error-aware compilation," *ACM JETC*, vol. 16, no. 3, 2020.

[8] P. Das *et al.*, "Jigsaw: Boosting fidelity of nisq programs via measurement subsetting," in *54th IEEE/ACM MICRO*, 2021.

[9] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for NISQ-era quantum devices," *Proc. of the 24th ACM ASPLOS*, 2019.

[10] Y. Nam *et al.*, "Automated optimization of large quantum circuits with continuous parameters," *npj Quantum Information*, vol. 4, no. 1, 2018.

[11] A. Cowtan *et al.*, "On the qubit routing problem," *in 14th TQC*, 2019.

[12] A. Zulehner *et al.*, "An efficient methodology for mapping quantum circuits to the IBM QX architectures," in *IEEE TCAD*, vol. 38, no. 7, 2019, pp. 1226–1236.

[13] A. Kissinger and J. van de Wetering, "Reducing the number of non-Clifford gates in quantum circuits," *Physical Review A*, vol. 102, no. 2, p. 022406, 2020.

[14] P. D. Nation *et al.*, "BenchPress: A scalable framework for quantum compiler benchmarking," *arXiv preprint arXiv:2404.13570*, 2024.

[15] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Adv. in NeurIPS*, vol. 30, 2017.

[16] The Qiskit Nature developers and contributors, "Qiskit nature 0.8.0." [Online]. Available: https://doi.org/10.5281/zenodo.7828768